

Stack (All-Questions)

(Question # 01)

```
class Stack
```

```
{
```

```
private:
```

```
int* ary;
```

```
int top;
```

```
int cap;
```

```
// data member
```

```
public:
```

```
Stack(int s);
```

```
{
```

```
// constructor
```

```
cap = s;
```

```
ary = new int[cap];
```

```
top = -1;
```

```
// initialize
```

```
}
```

```
~Stack()
```

```
{
```

```
// destructor
```

```
delete[] ary;
```

```
}
```

```
void push(int x)
```

```
{
```

```
// insert element
```

```
if (top == cap - 1)
```

```
{
```

```
// if cap is full
```

```
return;
```

```
}
```

```
top++;
```

```
ary[top] = x
```

```
// increment top
```

```
// assign top index new
```

```
}
```

```

void pop() // Delete element
{
    if (top == -1) // if no element
    {
        return;
    }
    prompt element; top--; // delete last entered element
}

```

```

void peek() // see last enter elem
{
    if (top == -1) // if no element
    {
        return;
    }
    prompt element; // cout element
}

```

```

bool is_empty() // checking emptiness.
{
    return top == -1; // return true if top == -1
}

```

```

void display() // display all elements
{
    if (top == -1) // if no element
    {
        return;
    }
    for (int i = top; i >= 0; i--) // run from last to 1st
    {
        prompt elements; // cout elements
    }
}

```

Question # 02

is explained in basic operations of Stack
(DRY-RUN)

1) Suppose: user push 1:

Node* newnode = new Node(1);

newnode->next = top;

top = newnode;

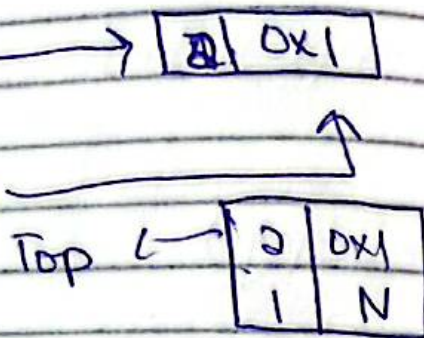


2) user push 2:

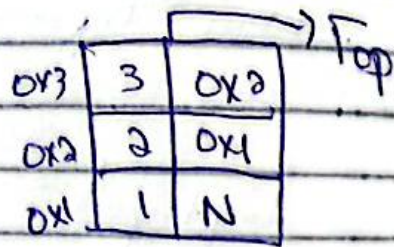
Node* newnode;

newnode->next = top;

top = newnode;



3) user push 3:



4) Now: peek

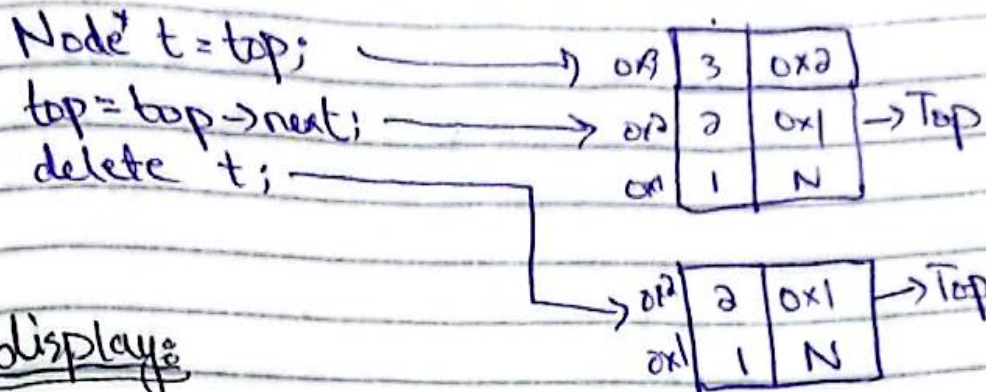
if (top == Null) x top = 0x3

cout << top->data;

3

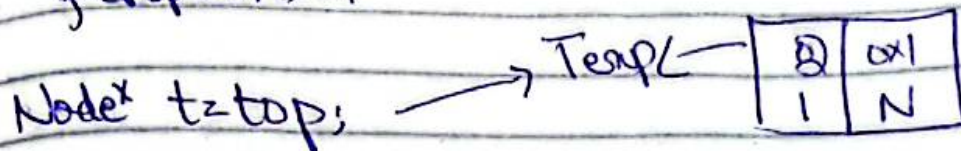
Now Pop:

if (top == Null) X top = 0x3



Now display:

if (top == N) X



1) while (t != N) ✓

{
cout << t->d; [2]
t = t->next; 0x1
}

2) while (t != N) ✓

{
cout << t->d; [1]
t = t->next; t = N
}

3) while (t != N) X t = N

Question #03

// Remaining part is same

```
void insertatbottom(int n)
{
    if (isempty()) { return; }
    push(n);
}
```

// if no element

```
int t = pop();
insertatbottom(x);
push(temp);
}
```

// pop into t
// recursing
// push temp

```
void reverse()
```

// reverse func

```
{
    if (isempty()) { return; }
}
```

// if no element

```
int t = pop();
reverse();
insertatbottom(temp);
}
```

// pop into t
// recursing
// call func

(DRY RUN)

Suppose: our stack is :

2	0x1	-top
1	N	

reverse: (a)

if (—) X

```
int t = pop();    t = 2
reverse();
```

reverse: (b)

if (—) X

```
int t = pop();    t = 1
reverse();
```

reverse: (c)

if (empty) ✓
return

insertatbtm(temp); (1)

insertatbtm(list x): (d)

if (empty) ✗

int t = pop()

t = []

insertatbtm

insertatbtm(list x): (e)

is (empty) ✓

push(1);

return

0 | 1 | N

insertatbtm(temp); t = 2

insertatbtm(list x) (a.f)

if (empty) ✗

int t = pop() t = 1;

call insertatbtm;

insertatbtm;

is (empty)

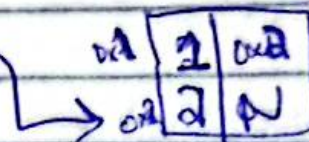
push(2);

return



push(1);

return;



Reverse List ↑

Question #04

```
void sortedinsert(int w)
{
    if (isempt || peak() < w)
    {
        push(w);
        return;
    }
    int t = pop();
    sortedinsert(w);
    push(t);
}
```

// if val > top element
// push val
// return to caller

// store pop into t
// recursive call
// push t

```
void sortstack()
{
    if (isempt)
    {
        return;
    }
    int t = pop();
    sortstack();
    sortedinsert(t);
}
```

// if empty

// return to caller

// store pop into t
// recursive call
// call function

(DRY RUN)

Suppose our stack is:

0x1	2	0x2
0x2	2	0x3
0x3	3	N

a) sortstack:

```
if (→) x
int t = pop()    t = 2
sortstack();
```

sortstack(2):

if (-) x

int t = pop() t = 1

sortstack;

sortstack(3):

if (-) x

int t = pop() t = 3

sortstack;

stack [10]

sortstack(4):

if (-) ✓

return; (to sortstack3)

sortedingcot(t); t = 3

sorted(t):

if (ep == 1) ✓

push(x);

return; (to stack)

3 | N

sorted(t)

t = 2

sorted(t):

if (-) ✓

push(x);

return (to stack 1)

1	0x3
3	N

sorted(t) t = 2:

sorted(t)

if (-) x

int t = pop

sorted(a)

t = 1

sorted(2):

if (-) ✓

push(t) →

return;

top sorted

2
3

push(t)

1
2
3

Question # 05

(DRY_RUN)

Suppose: our stack is

at	7	-
at	4	-
at	6	-

a) delete middle element:

if (—) ✗

int mid = size() / 2; mid = 3 / 2 = 1

del helper(0, mid);

①
del helper(curr, mid) curr = 0, mid = 1
if (—) ✗ 0 ≠ 1

int t = pop(); t = 7
del helper(curr + 1, mid);

②
del helper(curr, mid) curr = 1, mid = 1
if (—) ✓
{

pop(); 4 (poped)
return; (to dh 1)

push(top);

→

7
6

Question # 06

(DRY_RUN)

Suppose: our expression is { () }

is balanced

Stack S;

for (int i=0; i<str.length(); i++) ✓

ch = str[i];

ch = {

if (ch ==) ✓
{ s.push(ch); }
else if (—) ✗

{	N
---	---

for (—) ✓ 1 < 4

ch = str[i];

ch = {

if (ch ==) ✓
{ s.push(ch); }
else if (—) ✗

{	{
{	N

for (—) ✓ 2 < 4

ch = str[i];

ch =)

if (—) ✗
else if (—) ✓
{

if (—) ✗

char topch = s.pop();

topch = {

if (—) ✗

Stack →

{	N
---	---

for (—) ✓ 3 < 4

ch = str[i];

ch = }

if (—) ✗

else (—) ✓

{ if (—) ✗

char topch = s.pop();

topch = {

if (—) ✗

Question # 06]

(DRY_RUN)

Suppose

o) nextGreaterElement (int arr[], int n):

Stack S

int result[n];

o) for (int i = n-1; i >= 0; i--) ✓ $n > 20$

while (!s.isEmpty() && s.peek() < arr[i]) ✗

if (s.isEmpty()) ✓

res[i] = -1; }

else (—) ✗

s.push(arr[i])

→ arr[i] = 3

res[2] = -1

3

o) for (—) ✓ $i > 20$

while (!s.isEmpty() && s.peek() < arr[i]) ✓

s.pop()

3 popped

if (s.isEmpty()) ✓

res[i] = -1 }

else (—) ✗

s.push(arr[i]);

← arr[i] = 4

res[i] = { —, -1, 1 }

4 | N

o) for (—) ✓ $i > 20$

while (—) α

if (—) α

else

res[i] = s.peek();

res[i] = {4, -1, -1}

s.push(x & y[i]); ?

}

for (—) α $\rightarrow \geq 0$

Question # 08

(DRY_RUN)

Suppose stack is:

bool isPal()

{

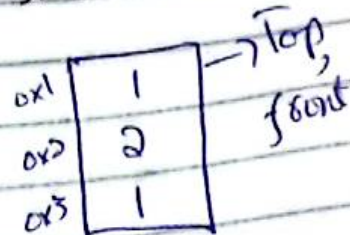
if (—) α

Node* front = top;

(Given)

f > 0x1

return cph(top, front);



bool cph(Node* cur, Node* F)

if (—) α

bool res = cph(c->next, F);

cur = 0x1, F = 0x1

(b)

bool cph(Node* c, Node* F)

c = 0x2, F = 0x1

if (—) α

bool res = cph(c->next, F);

(c)

bool cph(Node* c, Node* F)

c = 0x3, F = 0x1

if (—) α

bool res = cph(c->next, F);

1) bool cph(N¹ c, N² f)

if (—) ✓

return true;

// return to ②

if (—) ✗

bool checkz (cur->d = f->d) 2:1 ✓

front = front->next; f = 0x0

return check;

// return to b

if (—) ✗ checkz true

bool checkz (c->d == f->d); c = 0x0, f = 0x0
2:2 ✓

front = front->next;

↓
continue

if's palindrome
